

Reporte de Vulnerabilidad

Archivo: OrdersController.cs

Código Analizado:

```
?using Azure;
using FermaOrders.API.Controllers.Response;
using FermaOrders.Application.Dto.Application;
using FermaOrders.Application.Dto.Components.Customer;
using FermaOrders.Application.Interface.Application;
using FermaOrders.Domain.Entities;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using System.Net;

namespace FermaOrders.API.Controllers.Application
{
    [Route("api/[controller]")]
    [ApiController]
    public class OrdersController : ControllerBase
    {
        private readonly IOrderService _orderService;

        public OrdersController(IOrderService orderService)
        {
            _orderService = orderService;
        }

        [Authorize(Roles = "Admin, Empleado")]
        [HttpPost]
        [ProducesResponseType(StatusCodes.Status200OK)]
        [ProducesResponseType(StatusCodes.Status403Forbidden)]
        [ProducesResponseType(StatusCodes.Status500InternalServerError)]
        public async Task<IActionResult> CreateOrderAsync([FromBody] OrderDto orderDto)
        {
            var response = new RespuestaAPI();
            try
            {
                if (orderDto == null)
                {
                    return BadRequest("Invalid order data.");
                }

                var order = await _orderService.CreateOrderAsync(orderDto); // Aquí el await
                response.Result = order;
                response.IsSuccess = true;
                response.StatusCode = HttpStatusCode.OK;
                return Ok(response);
            }
            catch (Exception ex)
            {
                response.StatusCode = HttpStatusCode.InternalServerError;
                response.IsSuccess = false;
                response.ErrorMessages.Add($"Error: {ex.Message}");
                return StatusCode(500, response);
            }
        }

        [Authorize(Roles = "Admin, Empleado")]
        [HttpGet("orderitems/{orderId}")]
        [ProducesResponseType(StatusCodes.Status200OK)]
        [ProducesResponseType(StatusCodes.Status403Forbidden)]
        [ProducesResponseType(StatusCodes.Status500InternalServerError)]
        public async Task<IActionResult> GetOrderItemsByOrderIdAsync(int orderId)
        {
            var respuesta = new RespuestaAPI();
            try
            {
                var items = await _orderService.GetOrderItemByOrderAsync(orderId);

                respuesta.StatusCode = HttpStatusCode.OK;
                respuesta.IsSuccess = true;
                respuesta.Result = items;
            }
            catch (Exception ex)
            {
                respuesta.StatusCode = HttpStatusCode.InternalServerError;
                respuesta.IsSuccess = false;
                respuesta.ErrorMessages.Add($"Error: {ex.Message}");
                return StatusCode(500, respuesta);
            }
        }
    }
}
```

```

        return Ok(respuesta);
    }
    catch (KeyNotFoundException ex)
    {
        respuesta.StatusCode = HttpStatusCode.NotFound;
        respuesta.IsSuccess = false;
        respuesta.ErrorMessages.Add($"Error: {ex.Message}");
        return NotFound(respuesta);
    }
    catch (Exception ex)
    {
        respuesta.StatusCode = HttpStatusCode.InternalServerError;
        respuesta.IsSuccess = false;
        respuesta.ErrorMessages.Add($"Error: {ex.Message}");
        return StatusCode(500, respuesta);
    }
}

[Authorize(Roles = "Admin, Empleado")]
[HttpGet("customers/{customerId}")]
[ProducesResponseType(StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status403Forbidden)]
[ProducesResponseType(StatusCodes.Status500InternalServerError)]
public async Task<ActionResult> ListOrderByCustomer(int customerId)
{
    var respuesta = new RespuestaAPI();
    try
    {
        var orders = await _orderService.ListOrderByCustomer(customerId);

        respuesta.StatusCode = HttpStatusCode.OK;
        respuesta.IsSuccess = true;
        respuesta.Result = orders;

        return Ok(respuesta);
    }
    catch (KeyNotFoundException ex)
    {
        respuesta.StatusCode = HttpStatusCode.NotFound;
        respuesta.IsSuccess = false;
        respuesta.ErrorMessages.Add($"Error: {ex.Message}");
        return NotFound(respuesta);
    }
    catch (Exception ex)
    {
        respuesta.StatusCode = HttpStatusCode.InternalServerError;
        respuesta.IsSuccess = false;
        respuesta.ErrorMessages.Add($"Error: {ex.Message}");
        return StatusCode(500, respuesta);
    }
}
}
}
}
}

```

Análisis: ``html

Análisis de Vulnerabilidades y Calidad del Código

Posibles Vulnerabilidades

- **Vulnerabilidad:** Protección Insuficiente contra Inyección de Dependencias Maliciosas.

Línea Aproximada: Constructor de `OrdersController`:

```
public OrdersController(IOrderService orderService)
```

Descripción: Aunque el código usa inyección de dependencias (DI), no hay una validación explícita sobre la implementación de `IOrderService` que se inyecta. Un atacante podría, en teoría (dependiendo de cómo se configure la DI y se gestione el ciclo de vida de los objetos), registrar una implementación maliciosa de `IOrderService` que comprometa la aplicación.

Mitigación:

- Validar la implementación de `IOrderService` en el momento de la inyección (más difícil, pero posible con configuraciones de DI avanzadas).
- Asegurarse de que el contenedor de DI esté configurado de manera segura, evitando el registro de implementaciones no autorizadas de `IOrderService` en entornos de producción. Controlar estrictamente el acceso al código que configura el contenedor de DI.
- Aplicar el principio de mínimo privilegio. El código inyectado debe tener los mínimos permisos necesarios.

Mejora de Métricas: No directamente relacionado con las métricas de calidad estándar, pero mejora la seguridad, que es una dimensión importante de la calidad general del código.

- **Vulnerabilidad:** Falta de validación robusta de entrada en `ListOrderByCustomer`.

Línea Aproximada:

```
public async Task<IActionResult> ListOrderByCustomer(int customerId)
```

Descripción: El parámetro `customerId` es un entero que podría ser negativo o un valor extremadamente grande, lo que podría causar problemas en la consulta a la base de datos o en el servicio.

Mitigación:

- Validar el `customerId` para asegurar que sea un valor positivo y dentro de un rango razonable. Devolver un `BadRequest` si la validación falla.
- Considerar el uso de un tipo de dato más específico si el rango de `customerId` está bien definido.

Mejora de Métricas: Mejora la robustez del código y reduce la probabilidad de errores inesperados.

- **Vulnerabilidad:** Posible Denial of Service (DoS) por excepciones no manejadas.

Línea Aproximada: Bloques `catch` generales en todos los métodos.

Descripción: Capturar todas las excepciones (`catch (Exception ex)`) y simplemente retornar un error 500 puede enmascarar problemas subyacentes. Si la aplicación encuentra excepciones de forma continua (por ejemplo, por un ataque deliberado que intenta forzar excepciones), esto podría consumir recursos y resultar en un DoS.

Mitigación:

- Registrar adecuadamente la excepción (usando un logger).
- Considerar la implementación de un circuito breaker para prevenir que llamadas repetidas a un servicio que falla puedan tumbar la aplicación.
- Manejar excepciones específicas en lugar de capturar `Exception` de forma general.

Mejora de Métricas: Mejora la mantenibilidad y la observabilidad del código.

- **Vulnerabilidad:** Falta de paginación en `ListOrderByCustomer`.

Línea Aproximada:

```
var orders = await _orderService.ListOrderByCustomer(customerId);
```

Descripción: Si un cliente tiene una gran cantidad de órdenes, devolverlas todas en una sola respuesta puede ser ineficiente y consumir muchos recursos del servidor.

Mitigación:

- Implementar paginación en el método `ListOrderByCustomer`. Permitir que el cliente especifique el número de resultados por página y el número de página.

Mejora de Métricas: Mejora el rendimiento y la escalabilidad de la API.

Métricas de Calidad del Código

- **Complejidad Ciclomática:** Los métodos son relativamente simples, pero los bloques `try-catch` podrían aumentar la complejidad. Mejora: Refactorizar el manejo de excepciones en un middleware o filtro para evitar la duplicación de código.
- **Duplicación de Código:** Existe duplicación en el manejo de excepciones y en la construcción de la respuesta API en cada método. Mejora: Crear un método auxiliar o un filtro para manejar la construcción de

la respuesta API y el manejo de excepciones.

- **Legibilidad:** El código es relativamente legible, pero el uso de nombres inconsistentes (ej. `respuesta` vs `response`) dificulta la comprensión. Mejora: Estandarizar los nombres de las variables y los métodos en todo el código. Usar nombres en inglés consistentemente.
- **Acoplamiento:** El controlador depende de `IOrderService`. Este acoplamiento es necesario, pero se puede reducir aún más utilizando abstracciones adicionales si es necesario. Mejora: Asegurarse de que `IOrderService` defina una interfaz clara y concisa, y evitar que el controlador acceda directamente a detalles de implementación del servicio.

Solución Propuesta

Refactorización para Seguridad y Calidad

1. **Crear un Middleware para el Manejo de Excepciones:** Para evitar la duplicación del manejo de excepciones en cada método del controlador, se puede crear un middleware que capture todas las excepciones no manejadas y construya la respuesta de error adecuada. Esto centraliza el manejo de excepciones y reduce la duplicación de código.

2. **Crear un Helper para la Respuesta API:** Similar al manejo de excepciones, la construcción de la respuesta API (`RespuestaAPI`) es repetitiva. Un helper method o una clase abstracta base para los controladores puede encargarse de esto.

3. **Validación de Entrada con FluentValidation:** Utilizar FluentValidation para validar los objetos DTO (`OrderDto`) antes de pasarlos al servicio. Esto permite una validación más declarativa y fácil de mantener que la validación manual.

4. **Implementar Paginación:** Agregar parámetros para la paginación (`pageNumber`, `pageSize`) al método `ListOrderByCustomer` y al servicio correspondiente.

5. **Logger:** Usar un logger (Nlog, Serilog, etc) para registrar los errores y las excepciones de forma centralizada.

...